Finding Winning Lottery Tickets in Graph Neural Networks

Author:
Emiliana Ivanova Dimitrova
Sofia High School of Mathematics, Sofia, 11th grade

Under the scientific guidance of:
Radoslav Dimitrov
University of Oxford

Date:
March 2024

Abstract

Since the introduction of the Lottery Ticket Hypothesis, extensive research has been conducted on finding 'winning tickets' - sparse neural networks that achieve comparable performance while also using less storage and inference time than dense ones. Recent works have demonstrated the existence of such winning tickets and untrained sparse subnetworks at the initialization of graph neural networks. In our work, we estimate the probability of finding winning tickets in different types of graph neural networks.

1 Introduction

Through the years, neural networks have been growing rapidly in size to be able to solve more complex problems. They are being applied in various fields like Computer Vision, Natural Language Processing, Audio Processing, etc. However, this exponential growth in size has led to increased computational power requirements, memory usage, and training costs. [6]

Pruning has emerged as a popular technique to mitigate this issue by reducing the size of neural networks. This process involves training the model first and then deciding which parts can be eliminated. Among various pruning methods, our paper focuses on magnitude weights pruning [6], which removes a certain percentage of the lowest magnitude weights after a specific amount of training steps. While pruning effectively decreases the model's inference time and memory usage, it also raises intriguing questions about the necessity of the pruned information during the training process.

Investigating this, previous researchers took the approach of reinitializing the weights randomly and retraining them. However, this method led to a decrease in accuracy. The Lottery Ticket Hypothesis (LTH) [2], introduced in 2018, offered a novel approach: it reinitialized the pruned network's weights to their pre-training values, achieving comparable results to the original unpruned network's performance, even with a significant amount of pruning. This led to the formulation of the LTH, which states that:

A randomly initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.

The subnetwork mentioned above is also called a "winning ticket" and finding such subnetworks can decrease the storage requirements, improve the inference, and potentially the training time of the model.

Later the stronger version of the LTH [5] was proposed, suggesting that each overparameterized network contains a subnetwork with competitive accuracy without training. Such untrained winning tickets have been discovered for CNNs and GNNs. This prompted us to question the conditions under which winning tickets can be found and what the likelihood of finding such winning tickets is.

In the following sections, we will discuss our experiments and their results and explore the potential for future research in this area.

2 Related Work

2.1 Finding Winning tickets

[2] identifies winning tickets by first randomly initializing a dense neural network and training it, then pruning the network and resetting the remaining parameters to their initial weights from before the training (Figure 1).

Here is a more detailed explanation of the steps:

1. Randomly initialize a neural network $f(x;\theta_0)$ (where $\theta_0 \sim D_\theta$).
2. Train the network for j iterations, arriving at parameters θ_j.
3. Prune $p\%$ of the parameters in θ_j, creating a mask m.
4. Reset the remaining parameters to their values in θ_0, creating the winning ticket $f(x; m \odot \theta_0)$.

We denote a dense feed-forward neural network with $f(x;\theta_0)$, where θ_0 are the initial parameters. We also use a binary mask m $\in \{0,1\}^{|\theta|}$ for the weights.

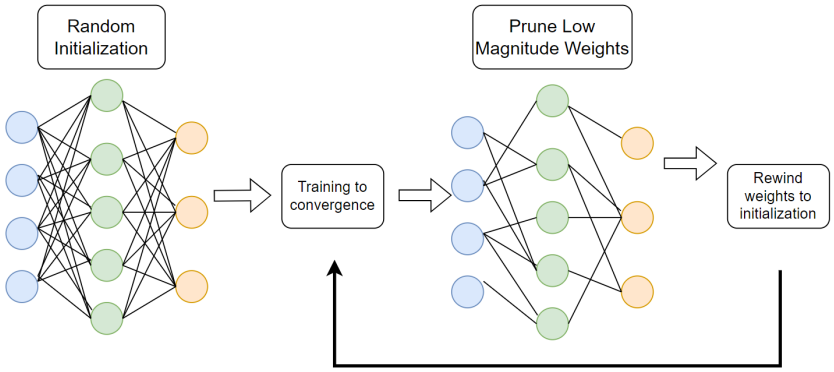

Figure 1: Iterative magnitude pruning

[2] finds winning tickets in a fully-connected architecture for MNIST and convolutional architectures for CIFAR10 which are 10-20% (or less) of the size of the original network.

2.2 Graph Neural Networks

Graph Neural Networks are neural architectures specifically designed for graph-structured data. They have been receiving significant attention recently and have found applications across different domains [8]. GNNs are used for tasks such as node classification, graph classification, and link prediction.

The key design element of GNNs is the concept of pairwise message passing. This process allows nodes in a graph to iteratively update their representations by sharing information

with their neighbors. This exchange of information is typically done using two key functions: AGGREGATE and COMBINE.

The AGGREGATE function (Figure 2), usually permutation invariant(Figure **??**), gathers information from the nodes in the neighborhood of the current node. The COMBINE function then combines this aggregated information with the current node's representation. This can be represented mathematically as:

$$a_v^k = \text{AGGREGATE}_k\{H_u^{k-1} : u \in N(v)\}$$

Here a_v^k denotes the aggregated version of the v-th node for the k-th layer of the graph neural network. H represents the hidden state of the node prior to aggregating and $N(v)$ signifies the neighborhood of node v.

$$H_v^k = \text{COMBINE}_k\{H_v^{k-1}, a_v^k\}$$

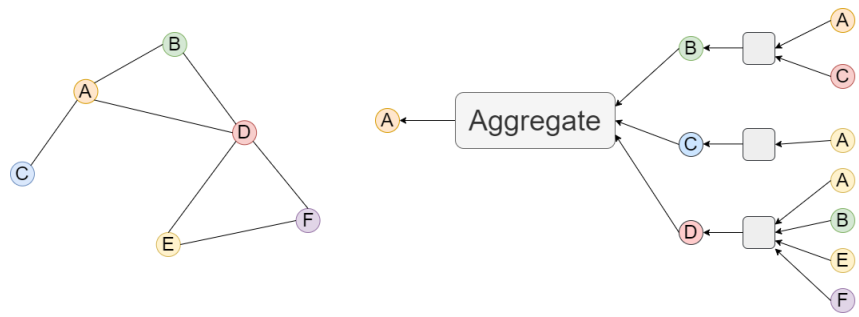

Figure 2: Graph Neural Networks

In our experiments we use 4 different types of graph neural networks - Graph Convolutional Networks with and without self-connections, Graph Isomorphism Networks and Graph Attention Networks.

2.2.1 Graph Convolutional Networks

Graph Convolutional Networks (GCN)[8] are an efficient variant of convolutional neural networks that operates directly on graphs, encoding both local graph structure and features of nodes. They can be expressed as:

$$H^{k+1} = ReLU\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^kW^k\right)$$

$\tilde{A} = A + I$ is the adjacency matrix of the given undirected graph with self-connections, which allows to include the node features itself when updating the node representations. $I \in \mathbb{R}^{N\times N}$ is the identity matrix. $\tilde{D}$ is a diagonal matrix with $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$.
The ReLU activation function is defined as $ReLU(x) = max(0, x)$.
$W_k \in \mathbb{R}^{F'\times F''}$ (where F', F'' are the dimensions of node representations in the k-th, $(k+1)$-th layer respectively) is a layerwise linear transformation matrix, which is trained during optimization.
We also conducted experiments, using Graph Convolutional Networks, but without self-connections:

$$H^{k+1} = \sigma\left(\tilde{D}^{-\frac{1}{2}}A\tilde{D}^{-\frac{1}{2}}H^kW^k\right)$$

A is the adjacency matrix.

2.2.2 Graph Isomorphism Networks

Introduced in 2019 [9], Graph Neural Networks have been shown to achieve significant discriminative power among GNNs. They are designed to capture the isomorphic properties of graphs, generalizing the Weisfeiler-Lehman (WL) test and thus effectively distinguish different graph structures. They calculate the representations of each node in the following way[8]:

$$H^{(k+1)} = MLP^{(k)}\left(((1+\epsilon^{(k)}) \cdot I + A) \cdot H^{(k)}\right)$$

$MLP^{(k)}$ stands for Multi-Layer Perceptron at the k-th layer. $\epsilon^{(k)}$ is a learnable parameter. It enables nodes in Graph Isomorphism Networks to differentiate their self-representation from those of their neighbors when updating.

2.2.3 Graph Attention Networks

Graph Attention Networks (GAT)[7] use the attention mechanism to compute the relationship between nodes and their neighbors.

First, we calculate attention coefficients that give information about the importance of node j's features to node i.

$$e_{ij} = LeakyReLU(a^T W h_i^{(k)} || W h_j^{(k)})$$

$.^T$ represents transposition and $||$ is the concatenation operation. W is the weight matrix, and $h_i^{(k)}$ is the vector, representing node i for the k-th layer of the neural network.
We then normalize the coefficients across all choices of j to make them easily comparable:

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})}$$

Finally, we compute the new representations of each node.

$$h_i^{(k+1)} = \sigma\left(\sum_{j \in N_i} \alpha_{ij} W h_j^{(k)}\right)$$

To obtain better representations of node features, we use multi-head attention. It involves executing M independent attention mechanisms for each layer of the GAT with different weight matrices and then getting the average as a result of the layer.

3 Experiments

We designed a synthetic task to test the probability of finding winning tickets in different types of graph neural networks. Moreover, we studied how the distance, from which information in graphs had to be collected, influenced the accuracy and the chances of finding winning tickets.

3.1 Synthetic task

For our experiments, we used the following synthetic task (Figure 3) :

We are given an undirected unweighted graph $G(V, E)$, where the vertices are either without color or with one out of C colors. Is there a node with the same color as node x at a distance $\leq D$?

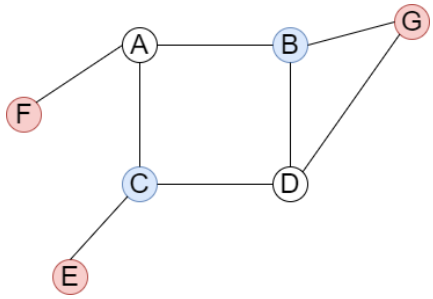

Figure 3: For node B and distance $D = 1$ the answer is False because there isn't another blue node at a distance ≤ 1. For node F and distance $D = 3$ the answer is True - nodes E and F are also red and at a distance 3.

3.2 Генериране на графи

Създадохме 5 набора данни за $1 \leq D \leq 5$, всеки от които съдържа 5000 случайни 3-regular графи със 100 върха и 2 цвята. Всеки връх е маркиран като True или False, в зависимост от това дали има връх със същия цвят на разстояние $\leq D$. За маркиране на върховете използваме алгоритъма за обхождане в ширина (BFS). Той започва от избран начален връх и изследва съседните му върхове на текущото разстояние, преди да премине към върховете на следващото разстояние. Алгоритмът използва опашка, за да следи върховете, които трябва да бъдат изследвани, като маркира всеки посетен връх, за да се избегне повторно посещение.

3.3 Experimental setup

Our goal was to estimate the probability of finding a winning ticket for different graph neural networks across different sparsities. To do that, we used iterative magnitude pruning, similar to [2]. First, for a fixed distance D, we trained our GNNs 36 times with random weights and computed the mean accuracy of the unpruned model μ_u. Then, we iteratively pruned the GNNs and calculated the mean μ_t and the standard deviation σ of the accuracy for each sparsity. Finally, we computed the z-score, which measures how many standard deviations the accuracy of a successful ticket is from the mean μ_u. The formula for the z-score is:

$$z = \frac{\mu_u - \mu_t}{\sigma}$$

Next, we computed the p-value using the cumulative distribution function (CDF) of the normal distribution. The p-value represents the probability that a random variable under the standard normal distribution is less than the z-score. However, since we are interested in the probability of a successful ticket (greater than the mean of the unpruned network μ_u), we subtract the result from 1:

$$p = 1 - \mathrm{CDF}(z) = 1 - \mathrm{stats.norm.cdf}(\mathrm{mu_true}, \mathrm{loc{=}mu_ticket}, \mathrm{scale{=}std_ticket})$$

This p-value then gives us the probability of finding a winning ticket.

For each distance D from 1 to 5, we trained each type of GNN with randomly initialized weights 36 times with 10 levels of pruning. We removed 30% of the weights at each level. Our GNNs had one input layer, 5 hidden layers with 64 neurons, and an output layer.
We used the SGD optimizer with a learning rate of 0.007, momentum of 0.9, and Cross Entropy Loss. Using a learning rate scheduler, the learning rate is reduced by a factor of 0.4 every 50 epochs. The model weights were initialized using the Kaiming Normal initialization method. For GATs we used multi-headed attention with 4 attention heads per layer.

We trained each level for 200 epochs before pruning it, with batch size 64 for GCNs and GINs and batch size 130 for GATs.

3.4 Results

In our experiments, we used four different types of graph neural networks - Graph Isomorphism Networks (GIN), Graph Attention Networks, and Graph Convolutional Networks (GCN) with and without self-connections.

Overall, it is evident from our results (Figure 4, 5) that the probability of finding a winning ticket in Graph Isomorphism Networks is much smaller than in Graph Convolutional Networks and Graph Attention Networks. GATs manage to find winning tickets with the highest probability for sparsity under 10%, which may indicate that the attention mechanism is more robust to pruning. Nevertheless, GINs achieve the best accuracy on our synthetic task, followed by GCNs without self-connection.

More specifically, the likelihood of finding a winning ticket in GINs starts between 10 and 15% at around 7% sparsity, then goes through numerous fluctuations, reaching between 35 and 50% for different distances at sparsity of 35% and later around 45% for 70% sparsity.

In contrast, GCNs with self-connections start at 7% for 7% sparsity, then rapidly increase to between 80 and 90% probability for 35% sparsity. However, for distances 4 and 1 the likelihood at sparsity 70% then decreases to roughly 65 and 70%, respectively.
GCNs without self-connections present a similar pattern. The main difference with those with self-connections is that the probability of finding winning tickets at 70% sparsity stays above 80% for all datasets.

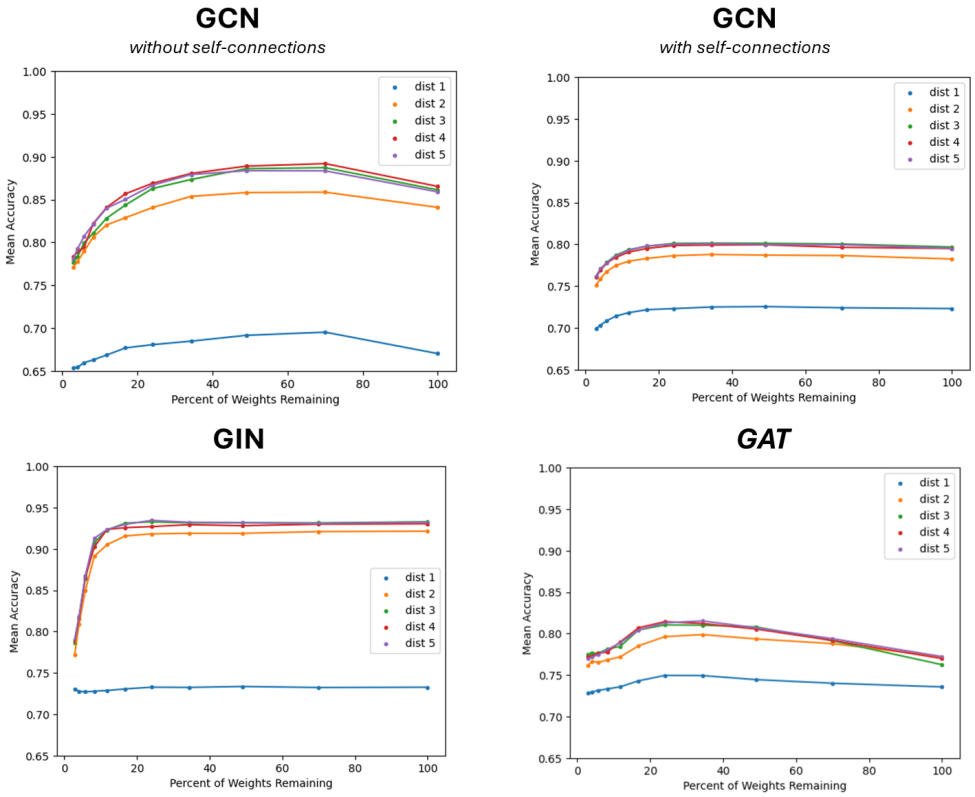

Figure 4: Probability of finding winning tickets across different sparsities and models for $1 \leq dist \leq 5$

GATs, on the other hand, start at between 20 and 70% chance of finding winning tickets for sparsities under 10%, reach a peak around 30% sparsity, and then decrease to between 60 and 85% probability of finding a winning ticket at 70% sparsity. Interestingly, as the distance increases, the likelihood of finding winning tickets for Graph Attention Networks also rises.

The second figure (Figure 5) shows the mean accuracy of the tickets for various sparsities. While GCNs without self-connections start at around 77% accuracy for 3% sparsity and distance above 1, and gradually increase to 87% accuracy for sparsity 70%, GINs start at 77% accuracy for 3% sparsity and rapidly increase to 93% accuracy for 15% sparsity and hit a plateau at this accuracy. Furthermore, GCNs with self-connections and GATs perform the worst - their mean accuracy is between 75% and 83% for distances more than 1 across all sparsities. Remarkably, the mean accuracy for distance 1 is much lower than for other distances for all models.

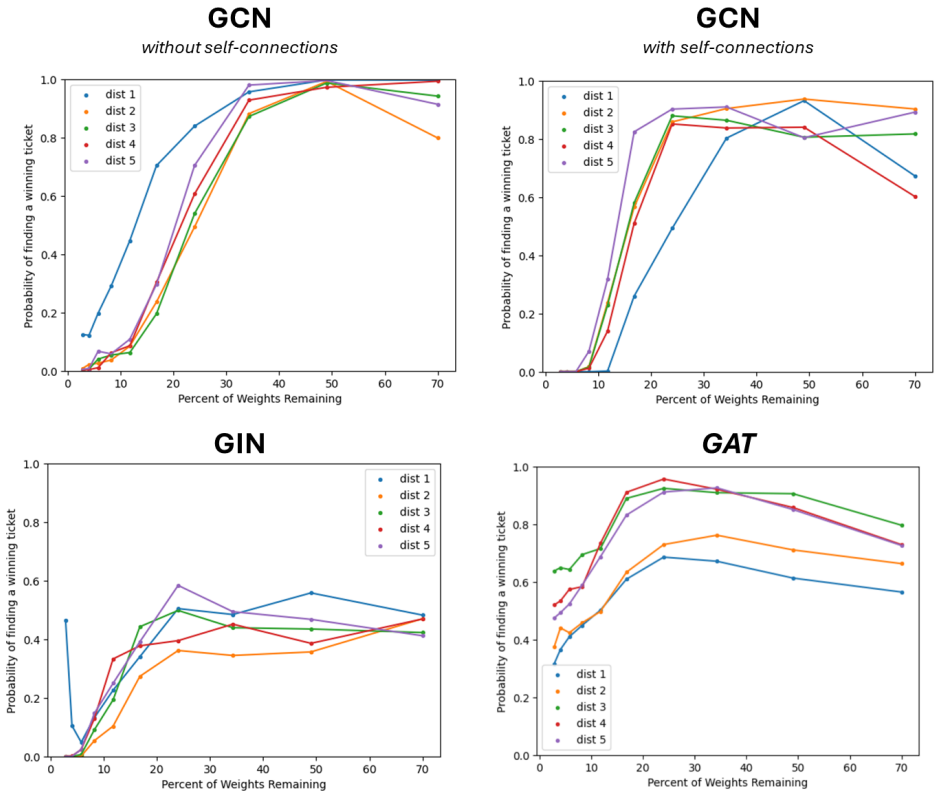

Figure 5: Mean accuracy of tickets with different sparsities for distance $1 \leq dist \leq 5$

3.5 Code

You can find our code and instructions to run it in this repository https://github.com/EmiDi3/finding_lth_in_gnns.git. To run our experiments, we used OpenLTH: A Framework for Lottery Tickets and Beyond [3].

4 Opportunities for further research

In the future, it would be interesting to further these experiments for larger datasets, and other types of graph neural networks such as GraphSAGE or other architectures such as Transformers. Another beneficial experiment would be measuring the probability of finding untrained winning tickets using different pruning techniques.

5 Conclusion

In this project, we evaluate how likely finding winning tickets is for different graph neural network architectures. The results can potentially enhance our understanding of the Lottery Ticket Hypothesis and the winning tickets, found in graph neural networks, as well as help us design more efficient neural networks with a higher probability of finding winning tickets within them.

6 Acknowledgements

For the development of this project, I want to thank, most importantly, my mentor Radoslav Dimitrov for his guidance and support throughout this project. I would like to thank HSSIMI for giving me the opportunity to conduct this research at the 23rd Summer Research School. I also want to thank Radostin Cholakov and Delyan Boychev for providing me with computational resources to conduct my experiments.

References

[1] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries, 2023.

[2] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks, 2019.

[3] Jonathan Frankle et al. Openlth: A framework for lottery tickets and beyond. https://github.com/facebookresearch/open_lth, 2019.

[4] Tianjin Huang, Tianlong Chen, Meng Fang, Vlado Menkovski, Jiaxu Zhao, Lu Yin, Yulong Pei, Decebal Constantin Mocanu, Zhangyang Wang, Mykola Pechenizkiy, and Shiwei Liu. You can have better graph neural networks by not training weights at all: Finding untrained gnns tickets, 2022.

[5] Vivek Ramanujan, Mitchell Wortsman, Aniruddha Kembhavi, Ali Farhadi, and Mohammad Rastegari. What's hidden in a randomly weighted neural network?, 2020.

[6] Sunil Vadera and Salem Ameen. Methods for pruning deep neural networks, 2021.

[7] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018.

[8] Lingfei Wu, Peng Cui, Jian Pei, and Liang Zhao. *Graph Neural Networks: Foundations, Frontiers, and Applications*. Springer Singapore, Singapore, 2022.

[9] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks?, 2019.